

04_advanced_models

June 5, 2026

1 04 — Modelos Avançados de Machine Learning

TCC PUC — Ciência de Dados e Big Data Evandro Ribeiro Gomes Coelho

Validação: TimeSeriesSplit (k=5) — mais adequado que k-fold aleatório do TCC2, pois respeita a ordem temporal dos dados. (Lembrar de citar isso no texto)

Métricas: MAE, RMSE, R^2 , MAPE

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import warnings
warnings.filterwarnings('ignore')

# Scikit-learn
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import MLPRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import TimeSeriesSplit, cross_val_score, GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

plt.rcParams['figure.figsize'] = (14, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3

PROC = '../data/processed/'

def compute_metrics(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    mask = y_true != 0
    mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100
    return {'Modelo': model_name, 'MAE': round(mae,4), 'RMSE': round(rmse,4),
```

```

        'R²': round(r2,5), 'MAPE (%)': round(mape,3)}

print('OK')

```

OK

```

[2]: # Carrega dados de features
df = pd.read_csv(PROC + 'df_features.csv', parse_dates=['Timestamp'])
df = df.sort_values('Timestamp').reset_index(drop=True)

# Features do TCC2 (reprodução)
FEATURES_TCC2 = [
    'Temperature', 'temp_lag1', 'energy_lag1',
    'hr_sin', 'hr_cos', 'is_holiday',
]

# Features expandidas (este TCC)
FEATURES_FULL = [
    'Temperature', 'temp_lag1', 'temp_lag2',
    'energy_lag1', 'energy_lag2', 'energy_lag24',
    'hr_sin', 'hr_cos',
    'dow_sin', 'dow_cos',
    'month_sin', 'month_cos',
    'roll_mean_6h', 'roll_mean_24h', 'roll_std_24h',
    'delta_temp', 'delta_energy', 'temp_x_hour',
    'is_weekend', 'is_holiday',
]
TARGET = 'Value'

# Split temporal 75/25
split_idx = int(len(df) * 0.75)
df_train = df.iloc[:split_idx].copy()
df_test = df.iloc[split_idx:].copy()

X_train_f = df_train[FEATURES_FULL].values
X_test_f = df_test[FEATURES_FULL].values
y_train = df_train[TARGET].values
y_test = df_test[TARGET].values

X_train_t = df_train[FEATURES_TCC2].values
X_test_t = df_test[FEATURES_TCC2].values

# TimeSeriesSplit para validação cruzada
tss = TimeSeriesSplit(n_splits=5)

print(f'Treino: {len(df_train)} | Teste: {len(df_test)}')

```

```

print(f'Features TCC2: {len(FEATURES_TCC2)} | Features full: {len(FEATURES_FULL)}')
print('TimeSeriesSplit(n_splits=5) configurado')

all_results = []

```

Treino: 12474 | Teste: 4158
 Features TCC2: 6 | Features full: 20
 TimeSeriesSplit(n_splits=5) configurado

1.1 1. MLP — Multilayer Perceptron

1.1.1 1a. Reprodução do TCC2

43 neurônios, tanh, adam, 1500 épocas, features originais do TCC2. Agora com TimeSeriesSplit ao invés de k-fold aleatório.

```

[3]: # MLP TCC2 (reprodução)
pipe_mlp_tcc2 = Pipeline([
    ('scaler', MinMaxScaler(feature_range=(-1, 1))),
    ('mlp', MLPRegressor(
        hidden_layer_sizes=(43,),
        activation='tanh',
        solver='adam',
        max_iter=1500,
        random_state=42,
    ))
])

# Cross-val com TimeSeriesSplit
cv_r2_tcc2 = cross_val_score(pipe_mlp_tcc2, X_train_t, y_train, cv=tss,
    scoring='r2')
print(f'MLP TCC2 - R² por fold (TimeSeriesSplit): {np.round(cv_r2_tcc2, 5)}')
print(f'MLP TCC2 - R² médio: {cv_r2_tcc2.mean():.5f} ± {cv_r2_tcc2.std():.5f}')

# Treina final e testa
pipe_mlp_tcc2.fit(X_train_t, y_train)
y_pred_mlp_tcc2 = pipe_mlp_tcc2.predict(X_test_t)
m = compute_metrics(y_test, y_pred_mlp_tcc2, 'MLP (TCC2 original)')
all_results.append(m)
print(f"Teste - MAE={m['MAE']:.4f} RMSE={m['RMSE']:.4f} R²={m['R²']:.5f}")

```

MLP TCC2 - R² por fold (TimeSeriesSplit): [-1.48778 -1.77863 -0.2299 -0.69617 -0.83678]
 MLP TCC2 - R² médio: -1.00585 ± 0.55786
 Teste - MAE=2435.5006 RMSE=2916.4034 R²=-0.10795

```

[4]: # MLP com features expandidas e GridSearchCV
pipe_mlp_new = Pipeline([

```

```

    ('scaler', MinMaxScaler(feature_range=(-1, 1))),
    ('mlp', MLPRegressor(activation='tanh', solver='adam', random_state=42))
])

param_grid_mlp = {
    'mlp__hidden_layer_sizes': [(43,), (50,), (100,), (50, 25)],
    'mlp__max_iter': [1500, 3000],
}

gs_mlp = GridSearchCV(
    pipe_mlp_new, param_grid_mlp,
    cv=tss, scoring='r2', n_jobs=1, verbose=0
)
gs_mlp.fit(X_train_f, y_train)

print(f'Melhores parâmetros MLP: {gs_mlp.best_params_}')
print(f'Melhor R2 (CV) MLP      : {gs_mlp.best_score_:.5f}')

y_pred_mlp_new = gs_mlp.predict(X_test_f)
m = compute_metrics(y_test, y_pred_mlp_new, 'MLP (features expandidas)')
all_results.append(m)
print(f"Teste - MAE={m['MAE']:.4f}  RMSE={m['RMSE']:.4f}  R2= {m['R2']:.5f}")

```

Melhores parâmetros MLP: {'mlp__hidden_layer_sizes': (100,), 'mlp__max_iter': 3000}

Melhor R² (CV) MLP : 0.57359

Teste - MAE=3.3644 RMSE=17.4877 R²=0.99996

1.2 2. Random Forest

Sugerido como trabalho futuro no TCC2.

```

[5]: pipe_rf = Pipeline([
    ('scaler', MinMaxScaler(feature_range=(-1, 1))),
    ('rf', RandomForestRegressor(random_state=42, n_jobs=-1))
])

param_grid_rf = {
    'rf__n_estimators': [100, 200],
    'rf__max_depth': [None, 10, 20],
    'rf__min_samples_split': [2, 5],
}

gs_rf = GridSearchCV(
    pipe_rf, param_grid_rf,
    cv=tss, scoring='r2', n_jobs=1, verbose=0
)
gs_rf.fit(X_train_f, y_train)

```

```

print(f'Melhores parâmetros RF: {gs_rf.best_params_}')
print(f'Melhor R2 (CV) RF      : {gs_rf.best_score_:.5f}')

y_pred_rf = gs_rf.predict(X_test_f)
m = compute_metrics(y_test, y_pred_rf, 'Random Forest')
all_results.append(m)
print(f'Teste - MAE={m['MAE']:.4f}  RMSE={m['RMSE']:.4f}  R2= {m['R2']:.5f}")

# Extrai feature importance
rf_best = gs_rf.best_estimator_.named_steps['rf']
fi_rf = pd.Series(rf_best.feature_importances_, index=FEATURES_FULL).
    ↪sort_values(ascending=False)
print('\nTop 10 features (Random Forest):')
print(fi_rf.head(10).to_string())

```

Melhores parâmetros RF: {'rf__max_depth': 20, 'rf__min_samples_split': 2, 'rf__n_estimators': 200}

Melhor R² (CV) RF : 0.98744

Teste - MAE=37.4732 RMSE=74.5665 R²=0.99928

Top 10 features (Random Forest):

energy_lag1	0.809007
delta_energy	0.185632
roll_mean_6h	0.003926
roll_mean_24h	0.000233
energy_lag2	0.000214
temp_x_hour	0.000129
energy_lag24	0.000128
roll_std_24h	0.000104
hr_cos	0.000100
Temperature	0.000077

1.3 3. XGBoost

```

[6]: # problemas de import no colab, testar localmente
try:
    import xgboost as xgb
    XGB_AVAILABLE = True
    print(f'XGBoost {xgb.__version__} disponível')
except ImportError:
    XGB_AVAILABLE = False
    print('XGBoost não instalado')

```

XGBoost 3.2.0 disponível

```

[7]: if XGB_AVAILABLE:
    from sklearn.pipeline import Pipeline as SKPipeline

```

```

pipe_xgb = Pipeline([
    ('scaler', MinMaxScaler(feature_range=(-1, 1))),
    ('xgb', xgb.XGBRegressor(
        objective='reg:squarederror',
        random_state=42,
        n_jobs=-1,
        verbosity=0,
    ))
])

param_grid_xgb = {
    'xgb__n_estimators': [100, 300],
    'xgb__max_depth': [4, 6],
    'xgb__learning_rate': [0.05, 0.1],
    'xgb__subsample': [0.8, 1.0],
}

gs_xgb = GridSearchCV(
    pipe_xgb, param_grid_xgb,
    cv=tss, scoring='r2', n_jobs=1, verbose=0
)

gs_xgb.fit(X_train_f, y_train)

print(f'Melhores parâmetros XGB: {gs_xgb.best_params_}')
print(f'Melhor R² (CV) XGB      : {gs_xgb.best_score_:.5f}')

y_pred_xgb = gs_xgb.predict(X_test_f)
m_xgb = compute_metrics(y_test, y_pred_xgb, 'XGBoost')
all_results.append(m_xgb)
print(f"Teste - MAE={m_xgb['MAE']:.4f}  RMSE={m_xgb['RMSE']:.4f}  □
↪R²={m_xgb['R²']:.5f}")

xgb_model = gs_xgb.best_estimator_.named_steps['xgb']
fi_xgb = pd.Series(xgb_model.feature_importances_, index=FEATURES_FULL).
↪sort_values(ascending=False)
print('\nTop 10 features (XGBoost):')
print(fi_xgb.head(10).to_string())
else:
    print('XGBoost não executado.')
    y_pred_xgb = None
    fi_xgb = None

```

Melhores parâmetros XGB: {'xgb__learning_rate': 0.05, 'xgb__max_depth': 4,
 'xgb__n_estimators': 300, 'xgb__subsample': 1.0}
 Melhor R² (CV) XGB : 0.98880
 Teste - MAE=95.8274 RMSE=128.1559 R²=0.99786

Top 10 features (XGBoost):

energy_lag1	0.757151
delta_energy	0.157323
roll_mean_6h	0.061603
temp_x_hour	0.007253
month_cos	0.004830
roll_mean_24h	0.001604
Temperature	0.001463
energy_lag24	0.001374
roll_std_24h	0.001300
month_sin	0.001177

1.4 4. LSTM — Long Short-Term Memory

```
[8]: # try except para testar localmente
try:
    import tensorflow as tf
    from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import LSTM, Dense, Dropout
    from tensorflow.keras.callbacks import EarlyStopping
    TF_AVAILABLE = True
    print(f'TensorFlow {tf.__version__} disponível')
except Exception as e:
    TF_AVAILABLE = False
    print(f'TensorFlow não disponível')
```

TensorFlow 2.21.0 disponível

```
[9]: if TF_AVAILABLE:
    # Normaliza os dados
    scaler_lstm = MinMaxScaler(feature_range=(-1, 1))
    X_train_scaled = scaler_lstm.fit_transform(X_train_f)
    X_test_scaled = scaler_lstm.transform(X_test_f)

    # Reshape para LSTM: (samples, timesteps=1, features)
    X_train_3d = X_train_scaled.reshape(X_train_scaled.shape[0], 1,
    ↪X_train_scaled.shape[1])
    X_test_3d = X_test_scaled.reshape(X_test_scaled.shape[0], 1, X_test_scaled.
    ↪shape[1])

    # Normaliza alvo
    scaler_y = MinMaxScaler(feature_range=(-1, 1))
    y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1)).ravel()

    # Arquitetura LSTM
    tf.random.set_seed(42)
    model_lstm = Sequential([
```

```

        LSTM(64, input_shape=(1, X_train_scaled.shape[1]),
↪return_sequences=False),
        Dropout(0.2),
        Dense(32, activation='relu'),
        Dense(1)
    ])
    model_lstm.compile(optimizer='adam', loss='mse', metrics=['mae'])
    model_lstm.summary()

    early_stop = EarlyStopping(monitor='val_loss', patience=10,
↪restore_best_weights=True)

    history = model_lstm.fit(
        X_train_3d, y_train_scaled,
        epochs=100,
        batch_size=128,
        validation_split=0.1,
        callbacks=[early_stop],
        verbose=1,
    )

    # Previsão
    y_pred_lstm_scaled = model_lstm.predict(X_test_3d).ravel()
    y_pred_lstm = scaler_y.inverse_transform(y_pred_lstm_scaled.reshape(-1, 1)).
↪ravel()

    m_lstm = compute_metrics(y_test, y_pred_lstm, 'LSTM')
    all_results.append(m_lstm)
    print(f"Teste - MAE={m_lstm['MAE']:.4f} RMSE={m_lstm['RMSE']:.4f} ↪
↪R²={m_lstm['R²']:.5f}")

    # Curva de aprendizado
    fig, ax = plt.subplots(figsize=(12, 4))
    ax.plot(history.history['loss'], label='Perda treino')
    ax.plot(history.history['val_loss'], label='Perda validação')
    ax.set_title('Curva de Aprendizado - LSTM')
    ax.set_xlabel('Época')
    ax.set_ylabel('MSE')
    ax.legend()
    plt.tight_layout()
    plt.show()
else:
    print('LSTM não executado.')
    y_pred_lstm = None

```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	21,760
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 23,873 (93.25 KB)

Trainable params: 23,873 (93.25 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

88/88 2s 3ms/step - loss: 0.0644 - mae: 0.1835 - val_loss: 0.0161 - val_mae: 0.1008

Epoch 2/100

88/88 0s 1ms/step - loss: 0.0173 - mae: 0.1017 - val_loss: 0.0099 - val_mae: 0.0760

Epoch 3/100

88/88 0s 1ms/step - loss: 0.0116 - mae: 0.0838 - val_loss: 0.0050 - val_mae: 0.0529

Epoch 4/100

88/88 0s 1ms/step - loss: 0.0073 - mae: 0.0674 - val_loss: 0.0020 - val_mae: 0.0360

Epoch 5/100

88/88 0s 1ms/step - loss: 0.0050 - mae: 0.0557 - val_loss: 0.0011 - val_mae: 0.0255

Epoch 6/100

88/88 0s 4ms/step - loss: 0.0039 - mae: 0.0489 - val_loss: 4.7746e-04 - val_mae: 0.0169

Epoch 7/100

88/88 0s 2ms/step - loss: 0.0034 - mae: 0.0455 - val_loss: 4.6316e-04 - val_mae: 0.0161

Epoch 8/100

88/88 0s 2ms/step - loss: 0.0029 - mae: 0.0419 - val_loss: 3.3442e-04 - val_mae: 0.0136

Epoch 9/100

88/88 0s 2ms/step - loss: 0.0026 - mae: 0.0395 - val_loss: 2.4021e-04 - val_mae: 0.0121

Epoch 10/100

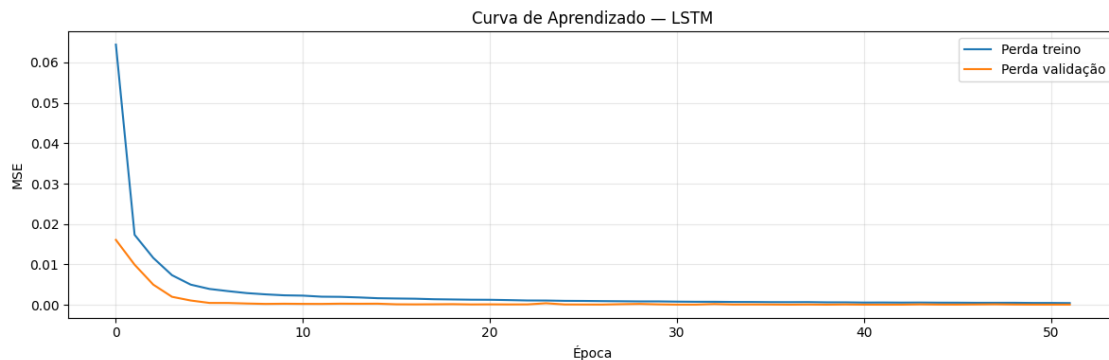
88/88 0s 2ms/step - loss:
 0.0024 - mae: 0.0378 - val_loss: 2.8186e-04 - val_mae: 0.0131
 Epoch 11/100
 88/88 0s 2ms/step - loss:
 0.0023 - mae: 0.0370 - val_loss: 2.4744e-04 - val_mae: 0.0122
 Epoch 12/100
 88/88 0s 2ms/step - loss:
 0.0020 - mae: 0.0348 - val_loss: 2.3614e-04 - val_mae: 0.0125
 Epoch 13/100
 88/88 0s 1ms/step - loss:
 0.0020 - mae: 0.0342 - val_loss: 2.9499e-04 - val_mae: 0.0144
 Epoch 14/100
 88/88 0s 2ms/step - loss:
 0.0018 - mae: 0.0327 - val_loss: 2.7580e-04 - val_mae: 0.0138
 Epoch 15/100
 88/88 0s 2ms/step - loss:
 0.0016 - mae: 0.0312 - val_loss: 2.9188e-04 - val_mae: 0.0134
 Epoch 16/100
 88/88 0s 2ms/step - loss:
 0.0016 - mae: 0.0304 - val_loss: 1.3733e-04 - val_mae: 0.0092
 Epoch 17/100
 88/88 0s 2ms/step - loss:
 0.0015 - mae: 0.0299 - val_loss: 1.1769e-04 - val_mae: 0.0085
 Epoch 18/100
 88/88 0s 1ms/step - loss:
 0.0014 - mae: 0.0285 - val_loss: 1.3910e-04 - val_mae: 0.0093
 Epoch 19/100
 88/88 0s 1ms/step - loss:
 0.0013 - mae: 0.0278 - val_loss: 1.7293e-04 - val_mae: 0.0103
 Epoch 20/100
 88/88 0s 1ms/step - loss:
 0.0013 - mae: 0.0273 - val_loss: 1.0544e-04 - val_mae: 0.0081
 Epoch 21/100
 88/88 0s 1ms/step - loss:
 0.0013 - mae: 0.0270 - val_loss: 1.2467e-04 - val_mae: 0.0086
 Epoch 22/100
 88/88 0s 1ms/step - loss:
 0.0012 - mae: 0.0262 - val_loss: 1.0315e-04 - val_mae: 0.0079
 Epoch 23/100
 88/88 0s 1ms/step - loss:
 0.0011 - mae: 0.0251 - val_loss: 1.1640e-04 - val_mae: 0.0083
 Epoch 24/100
 88/88 0s 1ms/step - loss:
 0.0011 - mae: 0.0248 - val_loss: 4.0024e-04 - val_mae: 0.0162
 Epoch 25/100
 88/88 0s 1ms/step - loss:
 9.9644e-04 - mae: 0.0239 - val_loss: 9.9391e-05 - val_mae: 0.0082
 Epoch 26/100

88/88 0s 1ms/step - loss:
 9.7489e-04 - mae: 0.0236 - val_loss: 8.4565e-05 - val_mae: 0.0072
 Epoch 27/100
 88/88 0s 1ms/step - loss:
 9.2928e-04 - mae: 0.0232 - val_loss: 7.3678e-05 - val_mae: 0.0066
 Epoch 28/100
 88/88 0s 1ms/step - loss:
 8.9446e-04 - mae: 0.0227 - val_loss: 1.6708e-04 - val_mae: 0.0105
 Epoch 29/100
 88/88 0s 1ms/step - loss:
 8.5402e-04 - mae: 0.0221 - val_loss: 2.3318e-04 - val_mae: 0.0134
 Epoch 30/100
 88/88 0s 1ms/step - loss:
 8.5624e-04 - mae: 0.0220 - val_loss: 1.2565e-04 - val_mae: 0.0098
 Epoch 31/100
 88/88 0s 1ms/step - loss:
 7.9778e-04 - mae: 0.0214 - val_loss: 6.3042e-05 - val_mae: 0.0063
 Epoch 32/100
 88/88 0s 1ms/step - loss:
 7.6498e-04 - mae: 0.0210 - val_loss: 6.6521e-05 - val_mae: 0.0066
 Epoch 33/100
 88/88 0s 1ms/step - loss:
 7.6291e-04 - mae: 0.0210 - val_loss: 2.0033e-04 - val_mae: 0.0117
 Epoch 34/100
 88/88 0s 1ms/step - loss:
 7.1813e-04 - mae: 0.0204 - val_loss: 8.2163e-05 - val_mae: 0.0074
 Epoch 35/100
 88/88 0s 2ms/step - loss:
 7.1098e-04 - mae: 0.0201 - val_loss: 9.3128e-05 - val_mae: 0.0071
 Epoch 36/100
 88/88 0s 1ms/step - loss:
 6.7702e-04 - mae: 0.0195 - val_loss: 8.2099e-05 - val_mae: 0.0068
 Epoch 37/100
 88/88 0s 1ms/step - loss:
 6.6984e-04 - mae: 0.0196 - val_loss: 4.8818e-05 - val_mae: 0.0054
 Epoch 38/100
 88/88 0s 2ms/step - loss:
 6.8218e-04 - mae: 0.0197 - val_loss: 8.0339e-05 - val_mae: 0.0068
 Epoch 39/100
 88/88 0s 2ms/step - loss:
 6.2531e-04 - mae: 0.0190 - val_loss: 4.5577e-05 - val_mae: 0.0050
 Epoch 40/100
 88/88 0s 1ms/step - loss:
 6.1920e-04 - mae: 0.0189 - val_loss: 9.0893e-05 - val_mae: 0.0079
 Epoch 41/100
 88/88 0s 1ms/step - loss:
 5.6892e-04 - mae: 0.0181 - val_loss: 4.6206e-05 - val_mae: 0.0053
 Epoch 42/100

```

88/88          0s 2ms/step - loss:
5.8370e-04 - mae: 0.0182 - val_loss: 4.0915e-05 - val_mae: 0.0049
Epoch 43/100
88/88          0s 1ms/step - loss:
5.6018e-04 - mae: 0.0179 - val_loss: 4.4579e-05 - val_mae: 0.0053
Epoch 44/100
88/88          0s 1ms/step - loss:
5.7373e-04 - mae: 0.0181 - val_loss: 1.1678e-04 - val_mae: 0.0086
Epoch 45/100
88/88          0s 1ms/step - loss:
5.4416e-04 - mae: 0.0177 - val_loss: 6.8590e-05 - val_mae: 0.0067
Epoch 46/100
88/88          0s 1ms/step - loss:
5.3857e-04 - mae: 0.0176 - val_loss: 5.9920e-05 - val_mae: 0.0062
Epoch 47/100
88/88          0s 1ms/step - loss:
5.1137e-04 - mae: 0.0171 - val_loss: 9.7976e-05 - val_mae: 0.0085
Epoch 48/100
88/88          0s 1ms/step - loss:
5.1548e-04 - mae: 0.0171 - val_loss: 1.3526e-04 - val_mae: 0.0085
Epoch 49/100
88/88          0s 1ms/step - loss:
5.1801e-04 - mae: 0.0173 - val_loss: 8.3004e-05 - val_mae: 0.0079
Epoch 50/100
88/88          0s 2ms/step - loss:
4.7655e-04 - mae: 0.0166 - val_loss: 5.8765e-05 - val_mae: 0.0064
Epoch 51/100
88/88          0s 2ms/step - loss:
4.7447e-04 - mae: 0.0164 - val_loss: 5.5328e-05 - val_mae: 0.0061
Epoch 52/100
88/88          0s 2ms/step - loss:
4.4373e-04 - mae: 0.0159 - val_loss: 6.0264e-05 - val_mae: 0.0066
130/130         0s 1ms/step
Teste - MAE=94.1410  RMSE=124.1225  R²=0.99799

```



1.5 5. Tabela Comparativa de Métricas

```
[10]: results_df = pd.DataFrame(all_results).sort_values('R²', ascending=False).
      ↪reset_index(drop=True)

print('=== COMPARAÇÃO DOS MODELOS AVANÇADOS ===')
display(results_df)

# Gráfico de barras comparativo
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

results_df_sorted_r2 = results_df.sort_values('R²', ascending=True)
results_df_sorted_rmse = results_df.sort_values('RMSE', ascending=False)

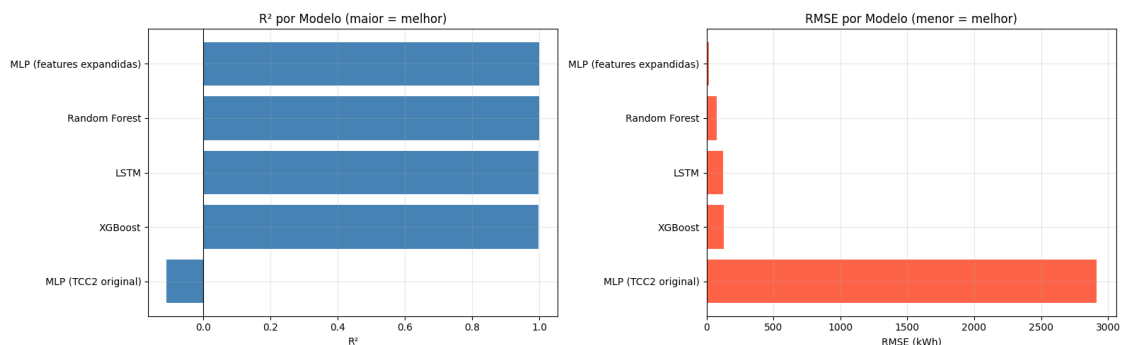
axes[0].barh(results_df_sorted_r2['Modelo'], results_df_sorted_r2['R²'],
             ↪color='steelblue')
axes[0].set_title('R² por Modelo (maior = melhor)')
axes[0].set_xlabel('R²')
axes[0].axvline(0, color='black', linewidth=0.8)

axes[1].barh(results_df_sorted_rmse['Modelo'], results_df_sorted_rmse['RMSE'],
             ↪color='tomato')
axes[1].set_title('RMSE por Modelo (menor = melhor)')
axes[1].set_xlabel('RMSE (kWh)')

plt.tight_layout()
plt.show()
```

=== COMPARAÇÃO DOS MODELOS AVANÇADOS ===

	Modelo	MAE	RMSE	R²	MAPE (%)
0	MLP (features expandidas)	3.3644	17.4877	0.99996	0.099
1	Random Forest	37.4732	74.5665	0.99928	0.988
2	LSTM	94.1410	124.1225	0.99799	3.636
3	XGBoost	95.8274	128.1559	0.99786	2.606
4	MLP (TCC2 original)	2435.5006	2916.4034	-0.10795	75.603



```
[11]: # Previsão vs Real (melhor modelo)
N_SHOW = 24 * 7
t_show = df_test['Timestamp'].values[:N_SHOW]
y_show = y_test[:N_SHOW]

best_model_name = results_df.iloc[0]['Modelo']
print(f'Melhor modelo: {best_model_name}')

pred_map = {
    'MLP (TCC2 original)': y_pred_mlp_tcc2,
    'MLP (features expandidas)': y_pred_mlp_new,
    'Random Forest': y_pred_rf,
}

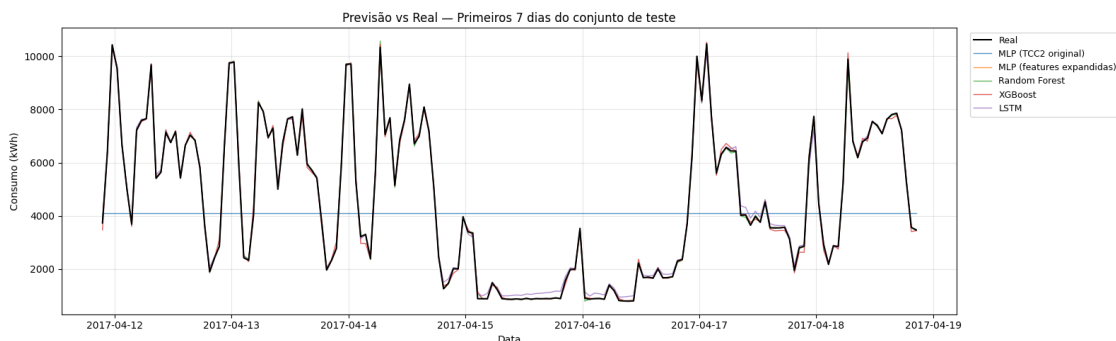
if XGB_AVAILABLE: pred_map['XGBoost'] = y_pred_xgb
if TF_AVAILABLE: pred_map['LSTM'] = y_pred_lstm

fig, ax = plt.subplots(figsize=(16, 5))
ax.plot(t_show, y_show, label='Real', color='black', linewidth=1.5, zorder=5)

for name, preds in pred_map.items():
    if preds is not None:
        ax.plot(t_show, preds[:N_SHOW], label=name, alpha=0.75, linewidth=1)

ax.set_title('Previsão vs Real - Primeiros 7 dias do conjunto de teste')
ax.set_xlabel('Data')
ax.set_ylabel('Consumo (kWh)')
ax.legend(bbox_to_anchor=(1.01, 1), loc='upper left', fontsize=9)
plt.tight_layout()
plt.show()
```

Melhor modelo: MLP (features expandidas)



```
[12]: # Salva resultados e predições
results_df.to_csv(PROC + 'results_advanced.csv', index=False)
print(f'Métricas salvas: {PROC}results_advanced.csv')

# Salva predições para o notebook 05
preds_df = df_test[['Timestamp', 'Value']].copy()
preds_df['pred_mlp_tcc2'] = y_pred_mlp_tcc2
preds_df['pred_mlp_new'] = y_pred_mlp_new
preds_df['pred_rf'] = y_pred_rf
if XGB_AVAILABLE and y_pred_xgb is not None:
    preds_df['pred_xgb'] = y_pred_xgb
if TF_AVAILABLE and y_pred_lstm is not None:
    preds_df['pred_lstm'] = y_pred_lstm
preds_df.to_csv(PROC + 'predictions.csv', index=False)
print(f'Predições salvas: {PROC}predictions.csv')

# Salva feature importances
fi_rf.to_csv(PROC + 'feature_importance_rf.csv', header=['importance'])
if XGB_AVAILABLE and fi_xgb is not None:
    fi_xgb.to_csv(PROC + 'feature_importance_xgb.csv', header=['importance'])
print('Feature importances salvas.')
```

Métricas salvas: ../data/processed/results_advanced.csv

Predições salvas: ../data/processed/predictions.csv

Feature importances salvas.